RETROSPECTIVA CICLO 5: REFACTORING Y EXTENSIÓN — CALIDAD CON VS CODE, JACOCO Y PMD

Escuela Colombiana de Ingeniería Julio Garavito — Desarrollo Orientado por Objetos (DOPO) | Periodo 2026-2

Proyecto: Simulador Slot Machine y Solución Problem I (49th ICPC World Championship)

Equipo de Desarrollo: Juan David Espitia y Cristian Salamanca

Dedicación reportada: 8 Horas/Hombre por integrante en el ciclo (16 Horas/Hombre en el ciclo; 80 Horas/Hombre acumuladas en los cinco ciclos)

Este documento continúa el informe consolidado de retrospectivas de los Ciclos 1, 2, 3 y 4 del proyecto Slot Machine y añade la retrospectiva del Ciclo 5 ("Refactoring y Extensión"). A diferencia del Ciclo 4, cuyo requisito dominante fue la extensibilidad con nuevos tipos de ruedas y de símbolos, el objetivo del Ciclo 5 fue evaluar y perfeccionar la calidad del simulador con herramientas modernas de desarrollo: migrar el proyecto a VS Code, medir el cubrimiento de las pruebas de unidad con JaCoCo (análisis dinámico, meta superior al 75 %) y verificar el cumplimiento de reglas de calidad de Java con PMD (análisis estático, meta de cumplir todas las reglas de prioridad alta). A continuación se exponen las decisiones tomadas, los resultados iniciales y finales de cada análisis, los problemas afrontados y los compromisos para la sustentación final.

RETROSPECTIVA CICLO 5: CALIDAD DEL SIMULADOR CON ANÁLISIS DINÁMICO Y ESTÁTICO

Alcance: Migración a VS Code, cierre y actualización del diseño en Astah, análisis dinámico con JaCoCo, análisis estático con PMD, refactorización derivada de los hallazgos, pruebas unitarias y prueba de aceptación.

1. ¿Cuáles fueron los mini-ciclos definidos? Justifíquenlos.

El Ciclo 5 no agregó funcionalidad nueva, por lo que el riesgo principal era que los cambios motivados por las herramientas rompieran lo construido en los ciclos anteriores. Por ello se definieron seis mini-ciclos que separan la migración, el diseño, cada análisis, la refactorización y las pruebas:

Mini-ciclo	Nombre	Alcance / Requisitos	Justificación Técnica y de Diseño
MC 5.1	Migración a VS Code	Traslado del proyecto a VS Code con las extensiones Extension Pack for Java y Test Runner for Java; estructura de carpetas .vscode, bin, lib, src y slotMachineBlueJ.	Las herramientas JaCoCo y PMD se ejecutan desde el entorno de VS Code, de modo que la migración era la base de los demás mini-ciclos. Se conservó la estructura proveniente de BlueJ (sin paquetes) para que la ejecución de las pruebas no generara problemas.
MC 5.2	Diseño en Astah	Actualización del diagrama de clases con la interfaz FindJackpot, SlotMachineContest como su implementación y las clases de prueba (SlotMachine*Test y SlotMachineAcceptanceTest); ajuste de nombres tras PMD.	Mantener la correspondencia estricta entre el diseño y el código: cualquier renombrado o cambio de firma motivado por PMD debía reflejarse también en los diagramas.
MC 5.3	Análisis dinámico con JaCoCo	Medición del cubrimiento inicial, estudio de las clases con menor porcentaje, creación de pruebas nuevas y medición del cubrimiento final. Meta: más del 75 %.	Medir primero y decidir después: los datos de JaCoCo orientaron qué clases y métodos probar en lugar de agregar pruebas al azar.
MC 5.4	Análisis estático con PMD	Ejecución de PMD con el ruleset quickstart.xml, clasificación de alertas por prioridad y corrección de las de prioridad alta (rojo).	Atender primero lo crítico (prioridad 1) y evaluar el costo de cada corrección de prioridad 3 frente al riesgo de afectar la estructura del proyecto y el diseño en Astah.

Mini-ciclo	Nombre	Alcance / Requisitos	Justificación Técnica y de Diseño
MC 5.5	Refactorización y limpieza	Variables en camelCase en SlotMachine, renombrado solve_base a solveBase, Canvas y CanvasPane como final, y limpiezas adicionales (importación sin uso, atributo type redundante en Wheel y return vacíos).	Aplicar cambios pequeños y verificables, cada uno validado con la suite completa en verde, para no introducir regresiones en los ciclos anteriores.
MC 5.6	Pruebas Unitarias y de Aceptación	Siete pruebas nuevas orientadas por JaCoCo, suites SlotMachine*Test (1, 2, 3, 4 y Contest) y SlotMachineAcceptanceTest ejecutándose en VS Code.	Cerrar el ciclo con evidencia: subir el cubrimiento y comprobar que la refactorización no alterara el comportamiento esperado.

En síntesis: Separar el análisis dinámico (MC 5.3) del estático (MC 5.4) y ambos de la refactorización (MC 5.5) permitió que cada cambio respondiera a un dato concreto de una herramienta y que la suite de pruebas validara cada paso.

2. ¿Cuál es el estado actual del proyecto en términos de mini-ciclos? ¿Por qué?

Al cierre del Ciclo 5, el proyecto reporta un 93 % de avance consolidado frente a los mini-ciclos planificados. Las metas de las dos herramientas se cumplieron; restan pendientes de prioridad baja y compromisos de ciclos anteriores:

Mini-ciclo	Nombre	Estado	Evidencias y Verificación del Estado
MC 5.1	Migración a VS Code	100 % Completado	Proyecto funcionando en VS Code con las extensiones instaladas; pruebas ejecutables desde Test Runner for Java.

Mini-ciclo	Nombre	Estado	Evidencias y Verificación del Estado
MC 5.2	Diseño Astah	90 % En Progreso	Diagrama de clases con FindJackpot y las clases de prueba, y renombrado a solveBase reflejado en los diagramas; quedan detalles puntuales de correspondencia por pulir.
MC 5.3	Análisis dinámico (JaCoCo)	100 % Completado	Cubrimiento total de 73.51 % a 91.22 %, superando la meta de 75 %.
MC 5.4	Análisis estático (PMD)	90 % En Progreso	De 13 alertas de prioridad 1 a 0 (meta cumplida); quedan 104 alertas de prioridad 3, que no se corrigieron de forma deliberada.
MC 5.5	Refactorización y limpieza	90 % En Progreso	Cambios aplicados con la suite en verde; siguen pendientes la extracción a métodos privados y el reemplazo de comparaciones de cadenas y casts.
MC 5.6	Pruebas Unitarias y Aceptación	90 % En Progreso	38 pruebas unitarias en verde, siete de ellas nuevas; SlotMachineAcceptanceTest con 91.67 % de cubrimiento; pendiente ensayar la prueba de aceptación para la sustentación.

¿Por qué se encuentra en este estado? Porque las dos metas medibles del ciclo (más del 75 % de cubrimiento y cero reglas rojas) están cumplidas y verificadas con las propias herramientas, pero se decidió reportar con transparencia, como porcentaje pendiente, lo que no quedó resuelto: las alertas de prioridad 3, el código repetido de SlotMachine y las clases con menor cubrimiento.

3. ¿Cuál fue el tiempo total invertido por cada uno de ustedes? (Horas/Hombre)

En el Ciclo 5 se mantuvo una inversión de 8 Horas/Hombre por integrante (16 Horas/Hombre en el ciclo y 80 Horas/Hombre acumuladas en los cinco ciclos) :

Desarrollador	Horas	Responsabilidades y Foco de Trabajo	Rol en el Ciclo
Juan David Espitia	8 Horas	Pruebas nuevas sobre Lefty, Wheel y SlotMachine (MC 5.3 y MC 5.6), refactorización de SlotMachine, FindJackpot y SlotMachineContest derivada de PMD (MC 5.5).	Ruedas y Núcleo de SlotMachine
Cristian Salamanca	8 Horas	Migración a VS Code (MC 5.1), diagramas en Astah (MC 5.2), análisis estático con PMD (MC 5.4), pruebas de Ephemeral y Shy y prueba de aceptación (MC 5.6).	Símbolos, Astah y Validaciones

4. ¿Cuál consideran fue el mayor logro? ¿Por qué?

Consideramos que el mayor logro del Ciclo 5 fue cumplir las dos metas de calidad del ciclo sin alterar la estructura ni el comportamiento del simulador: el cubrimiento de las pruebas de unidad pasó de 73.51 % a 91.22 % (meta: más del 75 %) y las alertas de prioridad alta de PMD pasaron de 13 a 0, con las 38 pruebas unitarias en verde después de la refactorización.

¿Por qué lo consideramos el mayor logro? Porque los números iniciales mostraban dónde estaban los puntos débiles y permitieron actuar sobre ellos. Clases como Lefty (de 42.65 % a 92.65 %), SlotMachineContest (de 63.64 % a 100 %), Wheel (de 65.86 % a 92.74 %) y SlotMachine (de 70.65 % a 91.80 %) quedaron cubiertas con siete pruebas nuevas: una Lefty bloqueada que no copia el símbolo de su vecina, Lefty en todos los modos de giro con y sin vecina, rueda normal bloqueada y desbloqueada en todos los giros, el ciclo de Ephemeral y Shy, métodos internos de Wheel, casos límite de SlotMachine en modo invisible y la invocación directa de solve(3). Además, estas pruebas permitieron ver errores que antes pasaban inadvertidos porque eran silenciosos y ninguna prueba los ejercitaba.

Pruebas añadidas a partir del análisis de JaCoCo:

Prueba añadida	Enfoque
shouldNotSpinLockedLeftyWheel	Comprueba que una rueda Lefty bloqueada no copie el símbolo de su vecina izquierda.
shouldSpinLeftyInAllModesWithAndWithoutNeighbor	Prueba Lefty en la posición uno sin vecina (giro aleatorio) y luego una segunda Lefty que copia a su vecina, activando el decrement del símbolo Ephemeral; además prueba su constructor parametrizado.
shouldLockAndUnlockNormalWheelInAllSpinModes	En las pruebas iniciales nunca se llamaba unlock ni se probaba una rueda bloqueada; ahora se comprueba que la rueda bloqueada se mantenga inmóvil en todos los giros.
shouldExerciseEphemeralShyAndSymbolOperations	Prueba el ciclo de los giros: visibilidad temporal y paso a estado invisible, el decrement de Ephemeral y que Shy alterne su visibilidad.
shouldExerciseWheelMethodsDirectly	Prueba métodos internos de las clases relacionadas con las ruedas.
shouldHandleBoundaryAndEmptyCasesInSlotMachine	Prueba las validaciones de SlotMachine en modo invisible (casos límite y máquina vacía).
solveDirectoYCasosLimite	Invoca directamente contest.solve(3), que antes no se llamaba en ninguna prueba.

SÍNTESIS DEL MAYOR LOGRO - CICLO 5

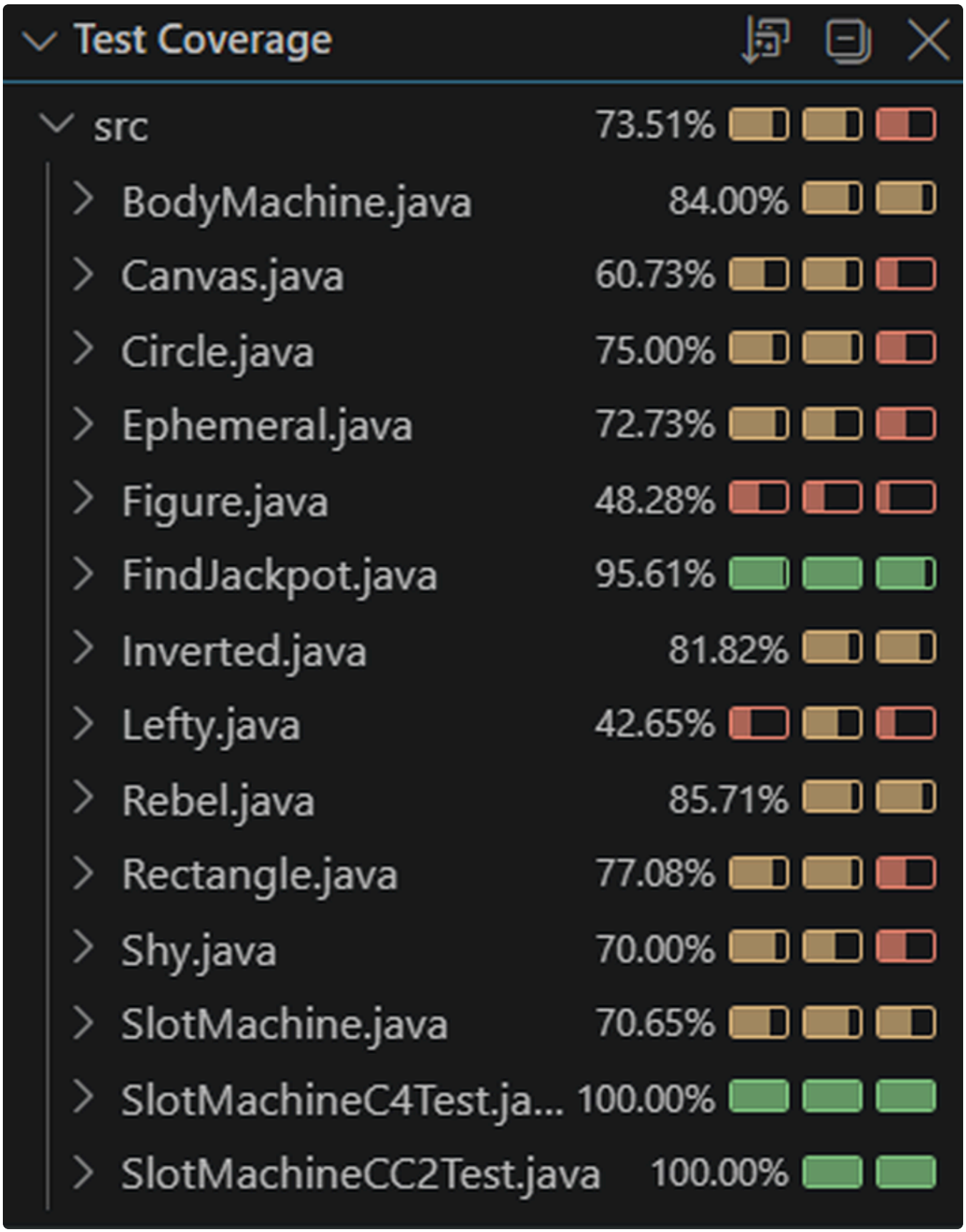
- Cubrimiento de pruebas de 73.51 % a 91.22 %, por encima de la meta del 75 %.
- Alertas de PMD de prioridad 1 de 13 a 0, con las 38 pruebas unitarias en verde tras la refactorización.
- Cambios hechos sobre datos de las herramientas y sin alterar las firmas ni el diseño de los ciclos anteriores.

Resultados de JaCoCo por clase (inicial → final):

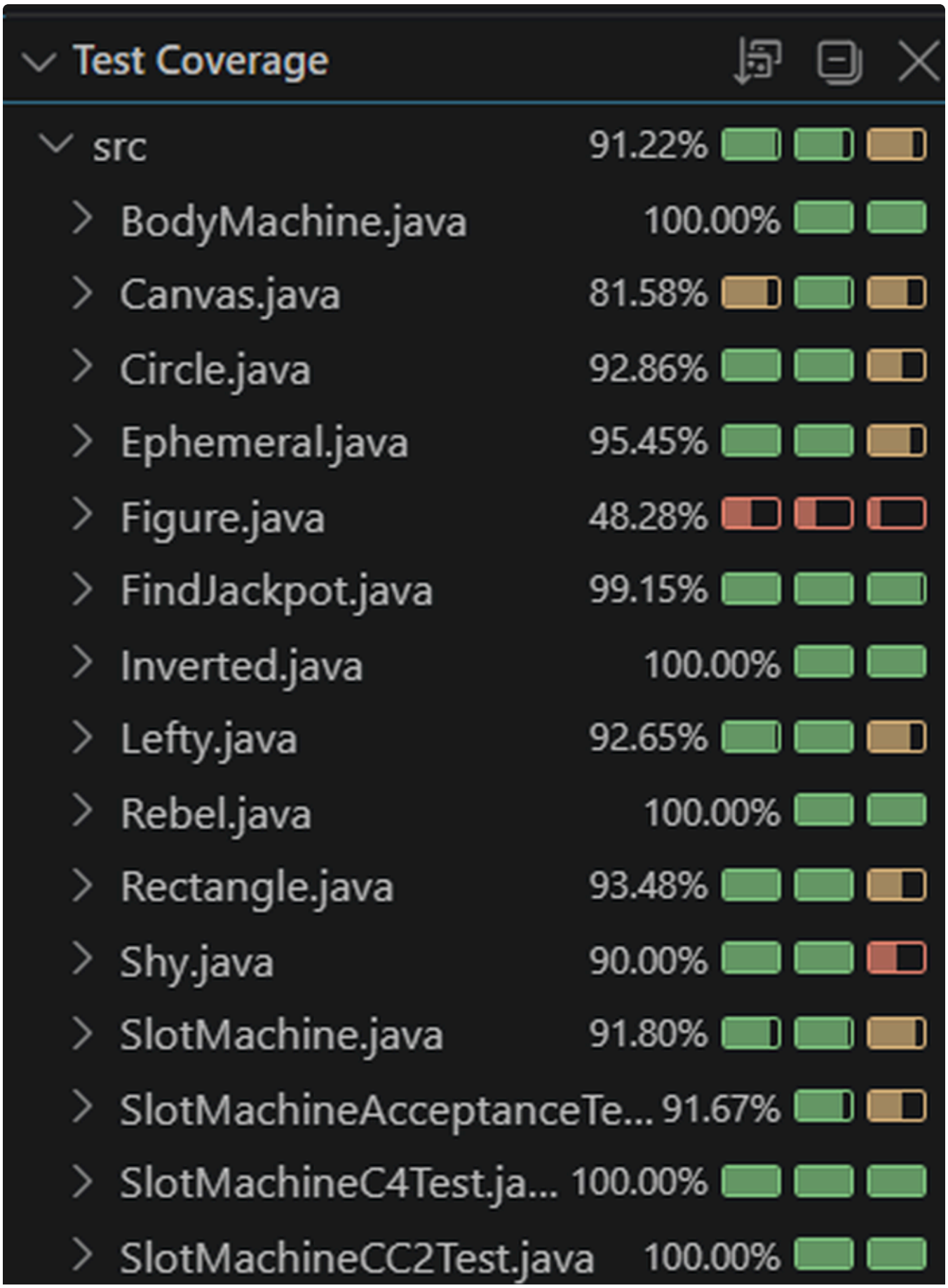
Clase	Inicial	Final
SlotMachineDialog	25.00 %	60.00 %
Lefty	42.65 %	92.65 %
Figure	48.28 %	48.28 %
Canvas	60.73 %	81.58 %
SlotMachineContest	63.64 %	100.00 %
Wheel	65.86 %	92.74 %
Triangle	68.00 %	86.00 %
Shy	70.00 %	90.00 %
SlotMachine	70.65 %	91.80 %
Ephemeral	72.73 %	95.45 %
Total del proyecto	**73.51 %**	**91.22 %**

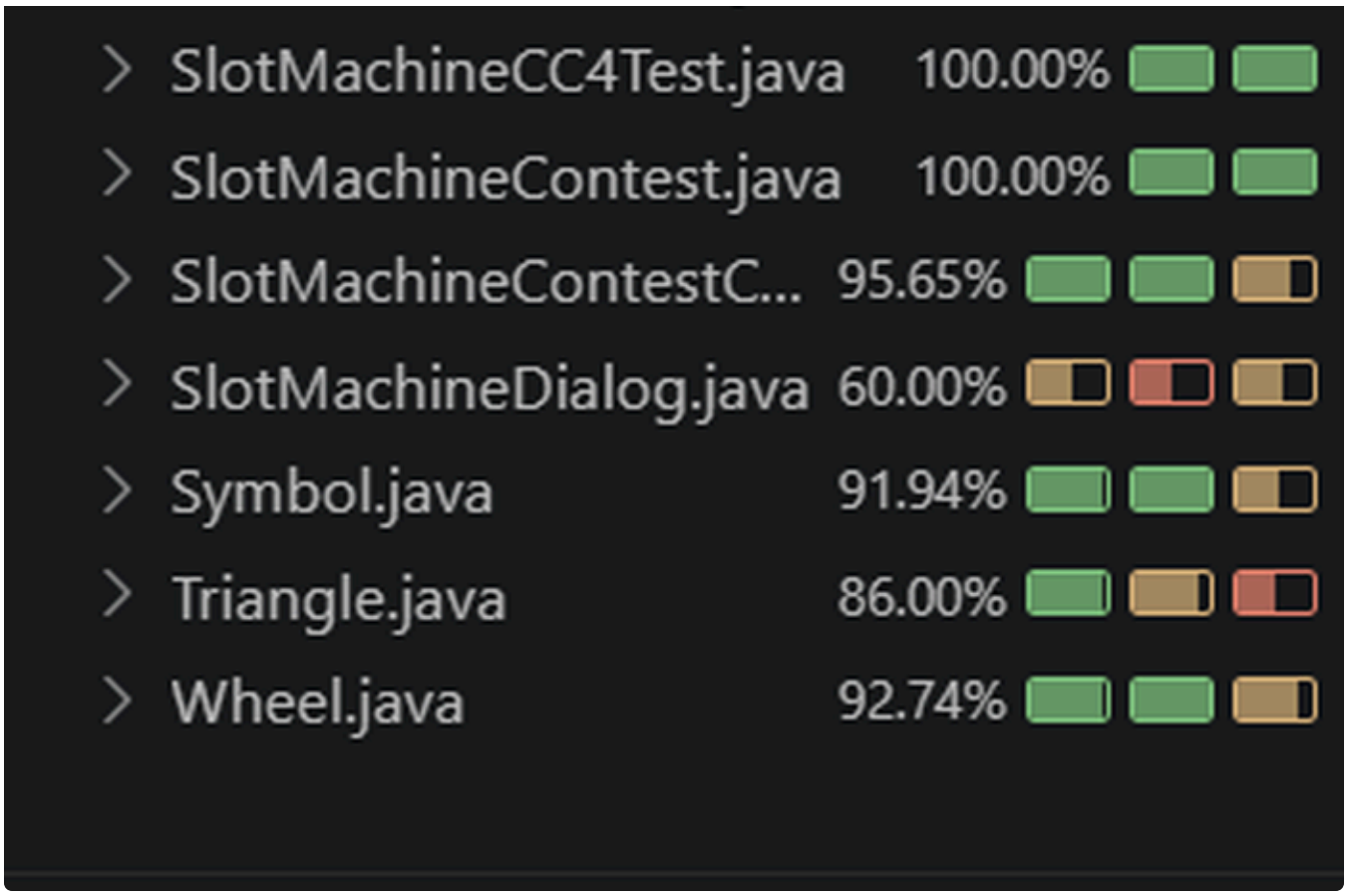
Por decisión de diseño no se cubrió con pruebas unitarias el método exit() de SlotMachine, porque ejecuta System.exit(0) y detendría JUnit y JaCoCo, ni se forzaron los diálogos de SlotMachineDialog, que están pensados exclusivamente para el modo visible. Esto explica el 60 % de esa clase.

Resultado inicial de JaCoCo (cubrimiento total del proyecto: 73.51 %):



Resultado final de JaCoCo (cubrimiento total del proyecto: 91.22 %):


Test Coverage
src 91.22%
BodyMachine.java 100.00%
Canvas.java 81.58%
Circle.java 92.86%
Ephemeral.java 95.45%
Figure.java 48.28%
FindJackpot.java 99.15%
Inverted.java 100.00%
Lefty.java 92.65%
Rebel.java 100.00%
Rectangle.java 93.48%
Shy.java 90.00%
SlotMachine.java 91.80%
SlotMachineAcceptanceTe... 91.67%
SlotMachineC4Test.ja... 100.00%
SlotMachineCC2Test.java 100.00%



5. ¿Cuál consideran que fue el mayor problema técnico? ¿Qué hicieron para resolverlo?

El mayor problema técnico del Ciclo 5 fue conciliar lo que pedían las herramientas con las restricciones de la estructura del proyecto y del diseño. PMD reportó 117 alertas (13 de prioridad 1 y 104 de prioridad 3), pero corregirlas todas implicaba tocar la estructura proveniente de BlueJ y modificar el diseño en Astah. Esto afectó, de nuevo, la reutilización de código y la elaboración de los diagramas.

PMD se usó desde VS Code con la extensión PMD for Java y el ruleset quickstart.xml, que clasifica las reglas por severidad así:

Severidad	Descripción
Prioridad 1 (rojo/error)	Problemas críticos de estándar o diseño.
Prioridad 3 (amarillo/warning)	Sugerencias de estilo.

La primera ejecución arrojó 117 alertas en total, 13 de prioridad alta y las demás de prioridad 3. Las de prioridad 1 correspondían a lo siguiente:

Cantidad	Clase	Descripción de la alerta
10	SlotMachine	Regla LocalVariableNamingConventions: variables locales en snake_case (principal_body, max_wheel, body_x, body_width y max_symbol).
2	Canvas	Regla ClassWithOnlyPrivateConstructorsShouldBeFinal: Canvas y CanvasPane tenían constructores privados pero no eran final.
1	FindJackpot	Regla MethodNamingConventions: el método solve_base tenía guion bajo en lugar de camelCase.

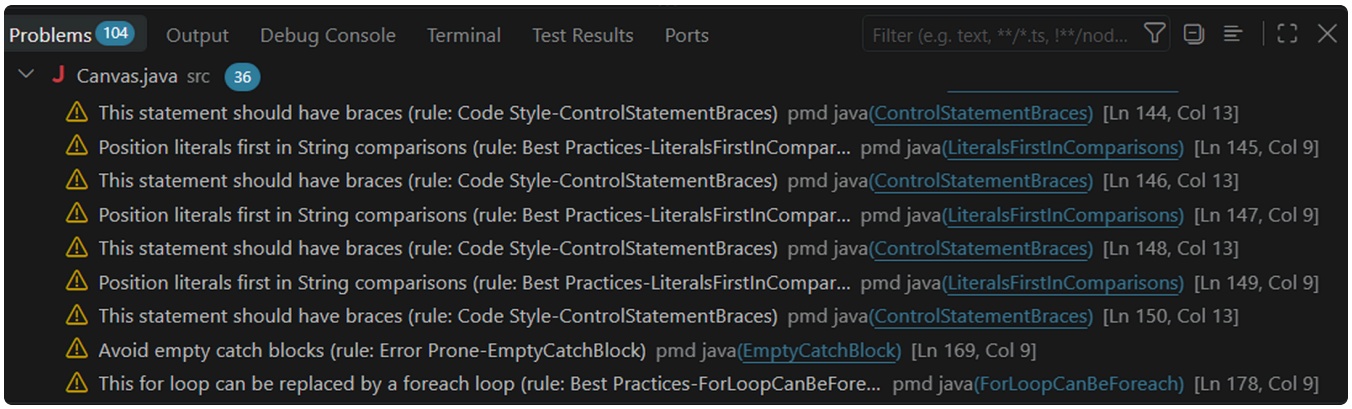
En cuanto al orden y la reutilización de código, las 13 alertas de prioridad 1 eran de tres tipos: 10 en SlotMachine por nombres de variables locales en snake_case (LocalVariableNamingConventions), 2 en Canvas y CanvasPane por tener constructores privados sin ser final (ClassWithOnlyPrivateConstructorsShouldBeFinal) y 1 en FindJackpot por el método solve_base con guion bajo (MethodNamingConventions). Se corrigieron todas, junto con limpiezas adicionales: se eliminó una importación de HashMap sin uso en SlotMachine, el atributo type redundante en Wheel y los return vacíos en Rebel y Symbol. Para la prioridad 3 se decidió no hacer mayores cambios, con el fin de no afectar la estructura proveniente de BlueJ ni generar problemas en la ejecución de las pruebas (por ejemplo, con el caso de NoPackage). Tampoco se agregaron métodos privados nuevos en SlotMachine, para mantener la correspondencia estricta con los métodos del diagrama de diseño en Astah. Reconocemos que esta decisión deja sin resolver la repetición ya identificada en el Ciclo 4 (reposicionamiento de ruedas, refresco de visibilidad y enlace de Lefty en varios métodos) y las comparaciones de cadenas de tipo con casts a Lefty.

Cambios realizados a partir de las alertas de prioridad 1:

Elemento	Cambio realizado
SlotMachine	Pasar todas las variables locales a camelCase (principalBody, maxWheel, bodyX, bodyWidth y maxSymbol).

Elemento	Cambio realizado
FindJackpot	Renombrar solve_base a solveBase en FindJackpot, SlotMachineContest, SlotMachineContestCC2Test y en los diagramas de Astah, quitando de paso el modificador public redundante en la interfaz.
Canvas y CanvasPane	Agregar final a Canvas y CanvasPane.

Resultado final de PMD (0 alertas de prioridad 1; el panel Problems muestra las 104 de prioridad 3, aquí las de Canvas.java):



En cuanto a los diagramas en Astah, el renombrado de solve_base a solveBase no era un cambio local: obligó a actualizar FindJackpot, SlotMachineContest, SlotMachineContestCC2Test y los diagramas de diseño, para que el código y el modelo siguieran coincidiendo. Se aprovechó para quitar el modificador public redundante en la interfaz.

Dimensión Técnica	Desafío y Dificultad Afrontada	Estrategia de Solución Aplicada
Orden y Reutilización de Código (PMD)	117 alertas, con restricciones de estructura (BlueJ, NoPackage) y de diseño (Astah) que impedían corregirlas todas, y código repetido de SlotMachine que seguía pendiente.	Corregir primero las 13 de prioridad 1 y las limpiezas seguras, validar con la suite completa en verde y dejar documentadas las de prioridad 3 y la extracción a métodos privados como mejora pendiente.

Dimensión Técnica	Desafío y Dificultad Afrontada	Estrategia de Solución Aplicada
Diagramas de Secuencia y de Clases (Astah)	Cada renombrado o limpieza del código debía reflejarse en el diseño para no perder la correspondencia entre ambos.	Actualizar en el mismo cambio el código, las pruebas y los diagramas (solveBase), sin agregar métodos al diseño que no estuvieran en él.
Cubrimiento de Pruebas (JaCoCo)	Métodos que no se pueden probar sin detener la ejecución (exit) o que son exclusivos del modo visible (diálogos).	Decisión explícita de no cubrirlos en pruebas unitarias y concentrar las siete pruebas nuevas en las clases con mayor brecha.

6. ¿Qué hicieron bien como equipo? ¿Qué se comprometen a hacer para mejorar los resultados?

¿Qué hicimos bien como equipo? Trabajamos con base en datos: medimos antes de decidir y repetimos la medición al final, lo que nos permitió demostrar la mejora del cubrimiento y de las reglas de PMD. Mantuvimos las 38 pruebas unitarias en verde después de cada refactorización, conservamos la correspondencia entre el código y el diseño en Astah (incluido el renombrado de solveBase) y documentamos las decisiones de lo que decidimos no cambiar y por qué.

¿A qué nos comprometemos para mejorar? Como cierre del proyecto y antes de la sustentación nos comprometemos a: (1) reportar de forma explícita los tipos desconocidos en addWheel y addSymbol, que hoy crean un elemento normal; (2) extraer a métodos privados el reposicionamiento de ruedas, el refresco visual y el enlace de Lefty, y reemplazar las comparaciones de cadenas y los casts por un mecanismo de creación por tipo, actualizando el diseño en Astah en consecuencia; (3) revisar las 104 alertas de prioridad 3 de PMD (por ejemplo, llaves en sentencias de control, literales primero en comparaciones de cadenas, bloques catch vacíos y bucles for convertibles a foreach), empezando por Canvas; (4) subir el cubrimiento de Figure (48.28 %) y de SlotMachineDialog (60 %); y (5) ensayar la prueba de aceptación en VS Code.

Dejamos constancia, además, de que los compromisos del Ciclo 3 de optimizar la heurística y de ejecutar pruebas de estrés con n=50 permanecen abiertos: el constructor SlotMachine(n) y FindJackpot admiten hoy valores de n entre 1 y 5.